

# Predictive Control of Heat Exchanger Using Koopman Model

Bc. Ivana Dukayová

Consultants: Ing. Michaela Horváthová, PhD.

Ing. Patrik Valábek

Institute of Information Engineering, Automation, and Mathematics

May 2025

# Motivation

- Energy efficiency
- Environmental impact
- Safety



# Project Goals

- Identification of a heat exchanger using:
  - Koopman operator
  - Strejc method

# Project Goals

- Identification of a heat exchanger using:
  - Koopman operator
  - Strejc method
- Design MPC for the heat exchanger

# Project Goals

- Identification of a heat exchanger using:
  - Koopman operator
  - Strejc method
- Design MPC for the heat exchanger
- Analyze control performance

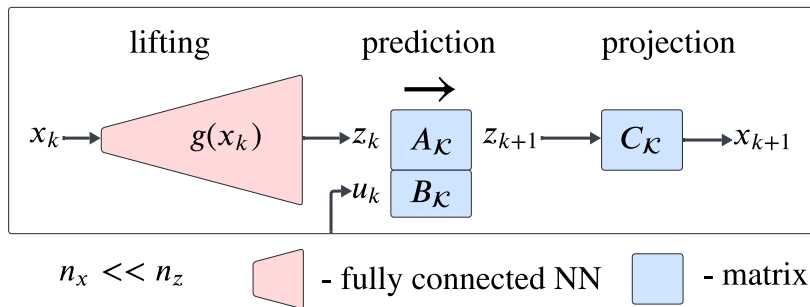
# Why Koopman?

- Linear models:
  - Simple and interpretable
  - Fail to capture nonlinear dynamics  $\rightarrow$  reduced performance

# Why Koopman?

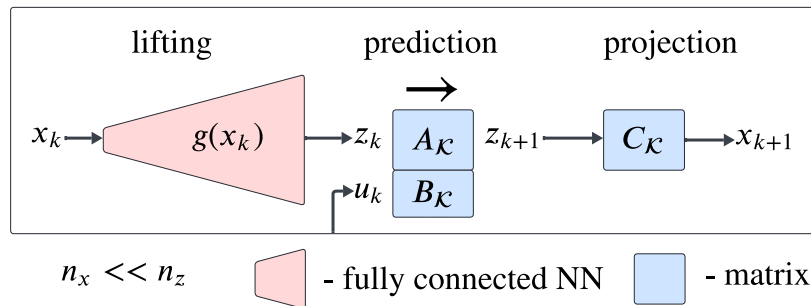
- Linear models:
  - Simple and interpretable
  - Fail to capture nonlinear dynamics  $\rightarrow$  reduced performance
- Koopman operator:
  - Uses neural networks to approximate nonlinear dynamics
  - Adds dimensions to states in which they are linear

# Koopman Identification Structure



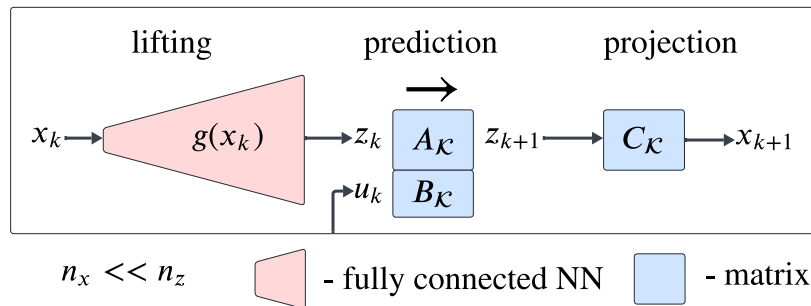


# Koopman Identification Structure



**Nonlinear dynamics:**  $x_{k+1} = f(x_k, u_k)$

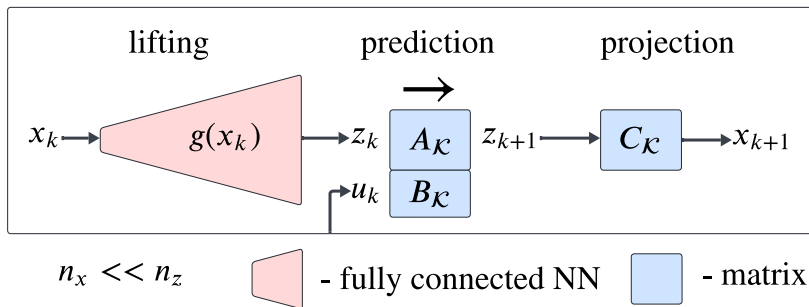
# Koopman Identification Structure



**Nonlinear dynamics:**  $x_{k+1} = f(x_k, u_k)$

**Mapping:**  $g(x_k)$ : maps measurements to lifted space  $z_k$

# Koopman Identification Structure

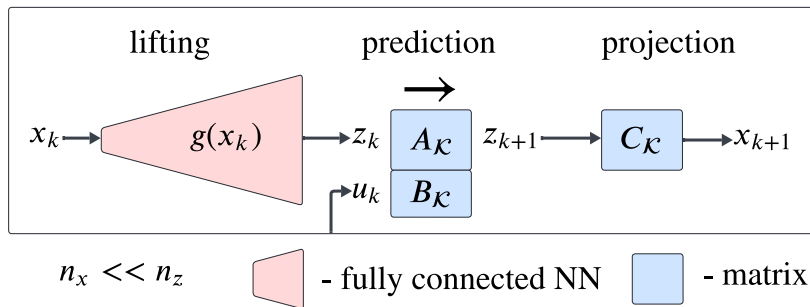


**Nonlinear dynamics:**  $x_{k+1} = f(x_k, u_k)$

**Mapping:**  $g(x_k)$ : maps measurements to lifted space  $z_k$

**Linear dynamics:**  $z_{k+1} = A_K z_k + B_K u_k$

# Koopman Identification Structure



**Nonlinear dynamics:**  $x_{k+1} = f(x_k, u_k)$

**Mapping:**  $g(x_k)$ : maps measurements to lifted space  $z_k$

**Linear dynamics:**  $z_{k+1} = A_K z_k + B_K u_k$

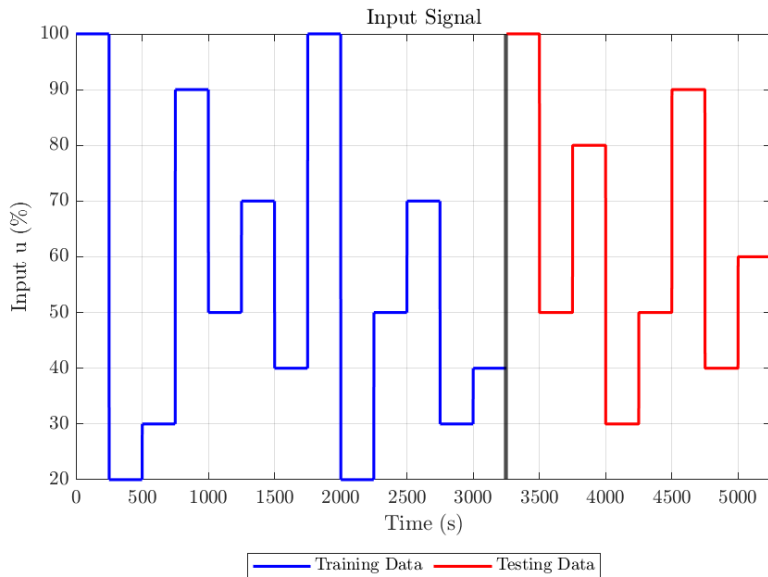
**Output:** predicts  $\hat{y}_k = C_K z_k$

# System Description

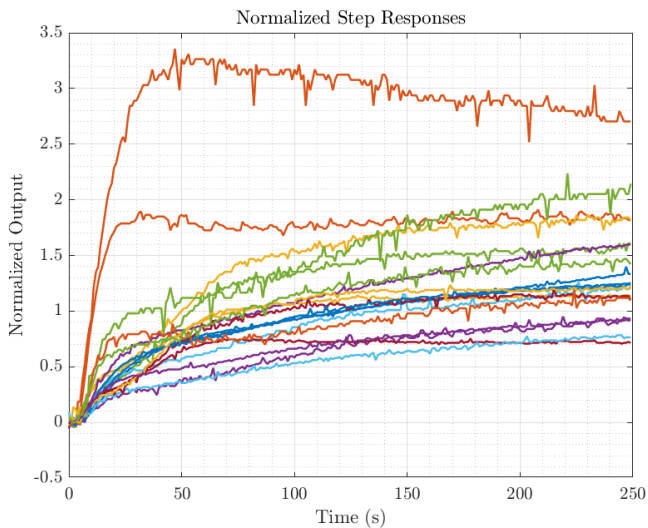
- Plate heat exchanger
- Input: heating pump speed (control signal in %)
- Output: outlet water temperature



# Applied Step Changes



# Measured Step Responses

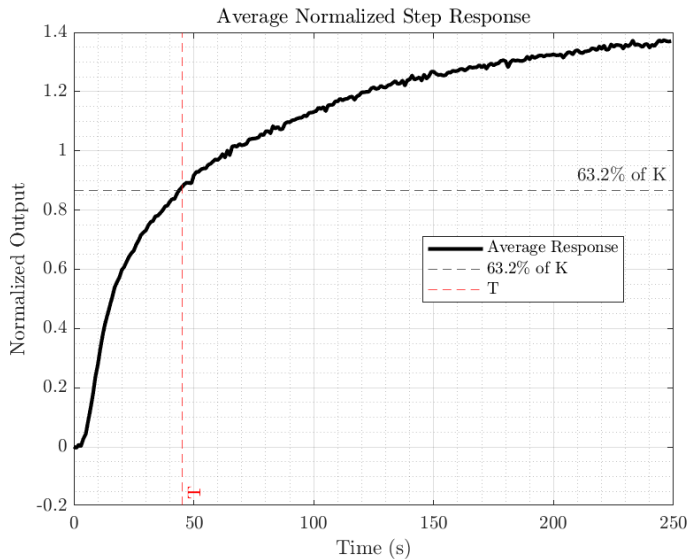


# Strejc Model Identification

- Input-output step response analysis
- Normalized responses based on unit step changes
- Parameters from average normalized step response



# Average Step Response Used for Control



# Strejc Model

Transfer function:

$$G(s) = \frac{1.37}{45s + 1}$$

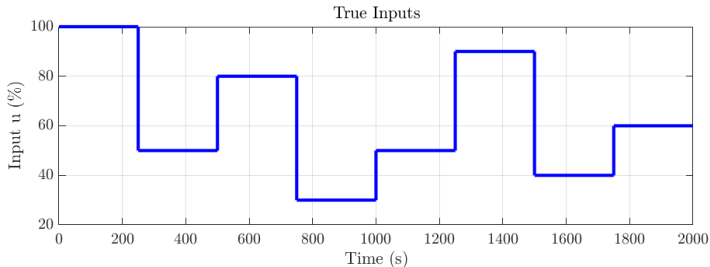
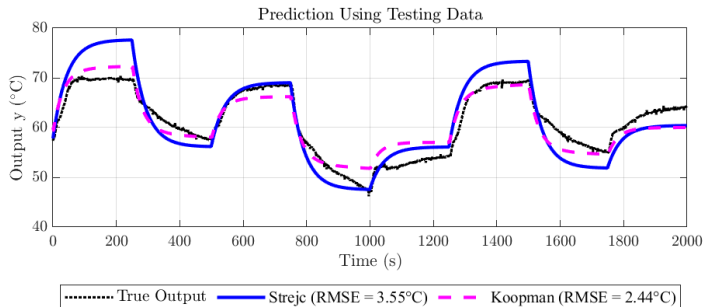
State space model:

$$A = 0.977 \quad B = 0.032 \quad C = 1 \quad D = 0$$

# Koopman Model Identification

- Koopman model:  $A_K \in \mathbb{R}^{10 \times 10}$ ,  $B_K \in \mathbb{R}^{10 \times 1}$ ,  $C_K \in \mathbb{R}^{1 \times 10}$
- Computed in Python using the Neuromancer, StandardScaler

# Strejc vs Koopman Prediction



# MPC Formulation and Parameters

## General MPC Problem:

$$\begin{aligned} \min_{u_0, \dots, u_{N-1}} \quad & \sum_{k=0}^{N-1} (x_k^\top Q_y x_k) + u_k^\top Q_u u_k \\ \text{s.t.} \quad & x_{k+1} = Ax_k + Bu_k \\ & y_k = Cx_k \\ & u_{\min} \leq u_k \leq u_{\max} \\ & y_{\min} \leq y_k \leq y_{\max} \\ & x_k = x_0 \end{aligned}$$

# MPC Formulation and Parameters

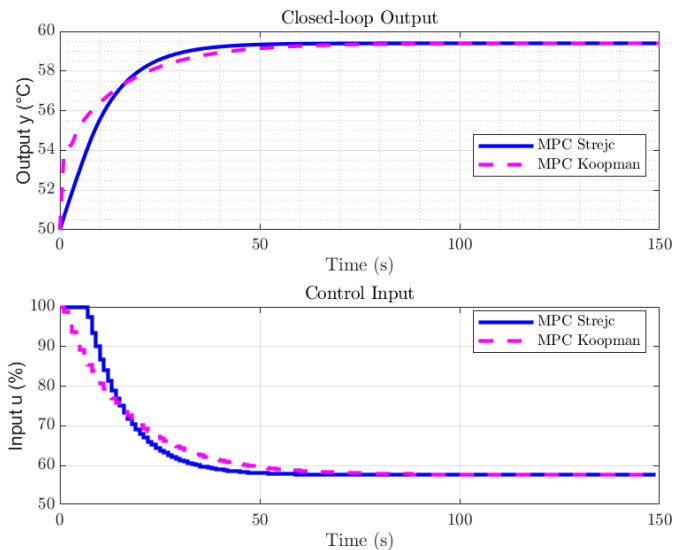
## General MPC Problem:

$$\begin{aligned} \min_{u_0, \dots, u_{N-1}} \quad & \sum_{k=0}^{N-1} (x_k^\top Q_y x_k) + u_k^\top Q_u u_k \\ \text{s.t.} \quad & x_{k+1} = Ax_k + Bu_k \\ & y_k = Cx_k \\ & u_{\min} \leq u_k \leq u_{\max} \\ & y_{\min} \leq y_k \leq y_{\max} \\ & x_k = x_0 \end{aligned}$$

## Used Parameters in Simulation:

- Prediction horizon:  $N = 40$
- Weights:  $Q_y = 10$ ,  $Q_u = 1$
- Constraints:  $u \in [20, 100]\%$ ,  $y \in [0, 70]^\circ\text{C}$
- Solved using YALMIP + quadprog

# Closed-loop Simulation



# Closed-loop Simulation

| Metric                    | Strejc | Koopman |
|---------------------------|--------|---------|
| Objective function value  | 111.49 | 72.30   |
| RMSE (open-loop) (°C)     | 3.55   | 2.44    |
| RMSE (closed-loop) (°C)   | 1.92   | 1.50    |
| Control effort $\sum  u $ | 9344.2 | 9360.9  |



# Closed-loop Simulation

| Metric                    | Strejc | Koopman |
|---------------------------|--------|---------|
| Objective function value  | 111.49 | 72.30   |
| RMSE (open-loop) (°C)     | 3.55   | 2.44    |
| RMSE (closed-loop) (°C)   | 1.92   | 1.50    |
| Control effort $\sum  u $ | 9344.2 | 9360.9  |

# Closed-loop Simulation

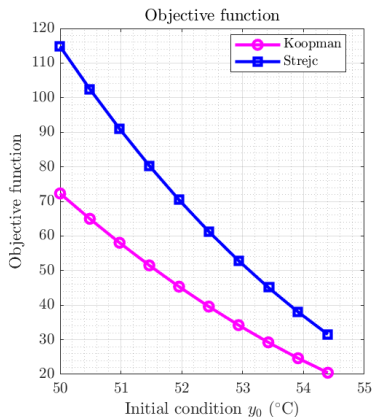
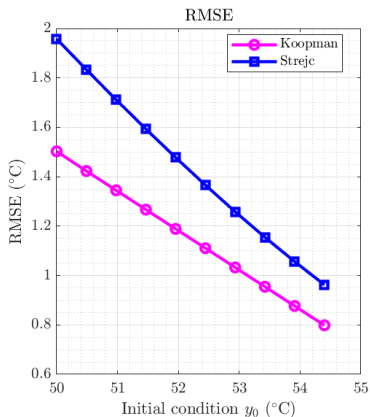
| <b>Metric</b>             | <b>Strejc</b> | <b>Koopman</b> |
|---------------------------|---------------|----------------|
| Objective function value  | 111.49        | 72.30          |
| RMSE (open-loop) (°C)     | 3.55          | 2.44           |
| RMSE (closed-loop) (°C)   | 1.92          | 1.50           |
| Control effort $\sum  u $ | 9344.2        | 9360.9         |

# Closed-loop Simulation

| Metric                    | Strejc | Koopman |
|---------------------------|--------|---------|
| Objective function value  | 111.49 | 72.30   |
| RMSE (open-loop) (°C)     | 3.55   | 2.44    |
| RMSE (closed-loop) (°C)   | 1.92   | 1.50    |
| Control effort $\sum  u $ | 9344.2 | 9360.9  |

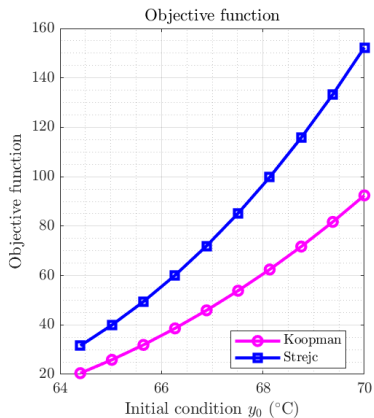
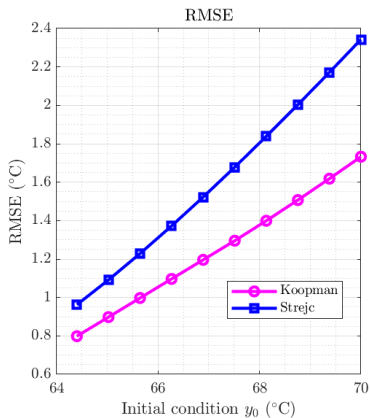
# Various Control Setups

## Control with Initial Conditions Below Steady-State



# Various Control Setups

## Control with Initial Conditions Above Steady-State



# Summary

- Identified system using Strejc and Koopman

# Summary

- Identified system using Strejc and Koopman
- Designed MPC for Strejc and Koopman

# Summary

- Identified system using Strejc and Koopman
- Designed MPC for Strejc and Koopman
- Control performance analyzed using simulations



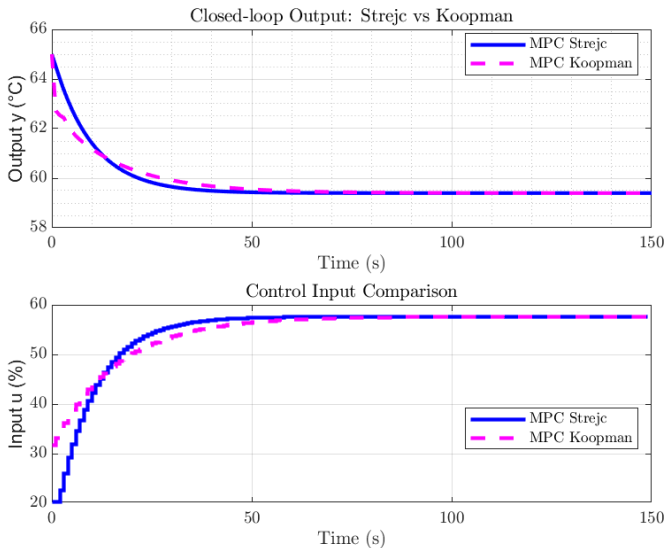
# Summary

- Identified system using Strejc and Koopman
- Designed MPC for Strejc and Koopman
- Control performance analyzed using simulations
- Experimental verification of the framework on a MIMO system

# Summary

- Identified system using Strejc and Koopman
- Designed MPC for Strejc and Koopman
- Control performance analyzed using simulations
- Experimental verification of the framework on a MIMO system
- Compare the Koopman model with more advanced data-driven method

# Closed-loop Performance from 65 °C



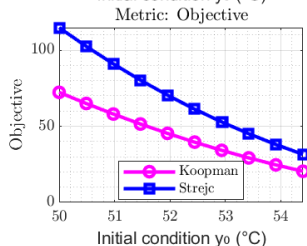
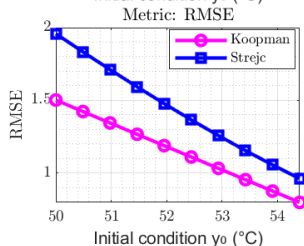
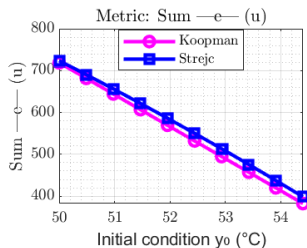
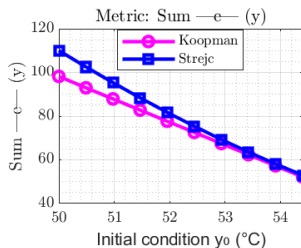
## Closed-loop Performance, from 65 °C

| Metric                    | Strejc  | Koopman |
|---------------------------|---------|---------|
| RMSE (open-loop) (°C)     | 3.55    | 2.44    |
| RMSE (closed-loop)(°C)    | 1.06    | 0.89    |
| Control effort $\sum  u $ | 8210.92 | 8214.9  |
| Objective function value  | 38.42   | 25.66   |

MPC (koopman)simulation time: 3.0509 seconds pre 150 samples ( $T_s = 1s$ )

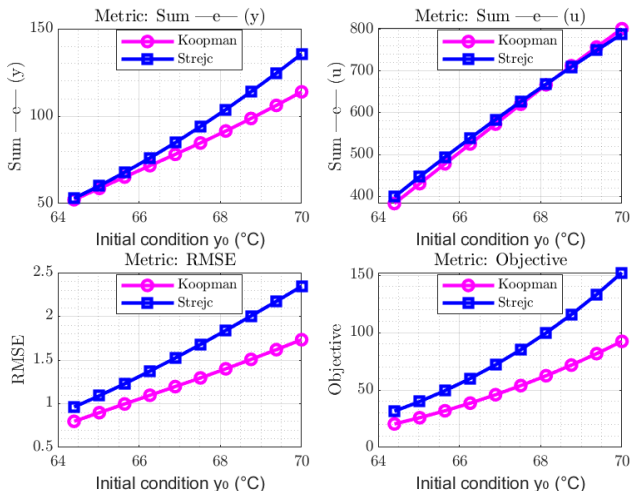
# Various Setups - More Metrics

## Koopman vs Strejc - Metrics for First 10 Initial Conditions



# Various Setups - More Metrics

## Koopman vs Strejc - Metrics for Last 10 Initial Conditions



# Koopman Model Structure (Neuromancer)

**Goal:** Identify a linear dynamic system in a learned latent space using neural networks.

- **Encoder**  $\phi(x)$  maps real system state  $x$  to latent state  $z$ :

$$z_k = \phi(x_k)$$

- **Koopman Dynamics:** Linear evolution in latent space:

$$z_{k+1} = Az_k + Bu_k$$

- **Decoder**  $\psi(z)$  reconstructs output from latent state:

$$\hat{y}_k = \psi(z_k)$$

- **Linear Output (used for MPC):**

$$y_k = Cz_k$$

- Trained on **scaled data (StandardScaler)** using train + test together (global scaling).

# Koopman Training Settings (Neuromancer)

**Framework:** Trained using the Neuromancer Trainer module with `PenaltyLoss`.

- **Epochs:** Early stopping at epoch 792
- **Final training loss:**

$$\text{train\_loss} = 0.3186$$

- **Latent dimension:** 10
- **Matrix sizes:**

$$A \in \mathbb{R}^{10 \times 10}, \quad B \in \mathbb{R}^{10 \times 1}, \quad C \in \mathbb{R}^{1 \times 10}$$

- **Dataset:** 5251 samples (3251 train + 2000 test)
- **Input scaling:**  $u \in [20\%, 100\%] \rightarrow$  standardized
- **Output:** temperature  $T_4 \in \mathbb{R}$



# Model Predictive Controller (Koopman) – Backup

$$\begin{aligned} \min_{u_0, \dots, u_{N-1}} \quad & \sum_{k=0}^{N-1} (y_k - \bar{y})^\top Q_y (y_k - \bar{y}) + \Delta u_k^\top Q_u \Delta u_k \\ \text{s.t.} \quad & z_{k+1} = Az_k + Bu_k \\ & y_k = Cz_k \\ & \Delta u_k = u_k - u_{k-1} \\ & u_{\min} \leq u_k \leq u_{\max} \\ & y_{\min} \leq y_k \leq y_{\max} \end{aligned}$$

**Given:**  $z_0 = \hat{z}_{k|k}, \quad d = \hat{d}_{k|k}, \quad u_{-1} = u_{\text{prev}}$

# Training Loss Composition (Koopman)

The total loss minimized during training was a weighted sum of 4 terms:

- **Output trajectory loss:**

$$y\_loss = 10 \cdot \text{MSE}(\hat{y}_{1:T}, y_{1:T})$$

Penalizes mismatch between predicted and true outputs over full horizon.

- **Latent dynamics consistency:**

$$x\_loss = 1 \cdot \text{MSE}(z_{1:T}, z_{koop,1:T})$$

Encourages agreement between encoder and Koopman-evolved latent states.

- **One-step prediction loss:**

$$\text{onestep\_loss} = 1 \cdot \text{MSE}(\hat{y}_1, y_1)$$

Ensures good local prediction.

- **Initial output reconstruction:**

$$\text{reconstruction\_loss} = 20 \cdot \text{MSE}(\hat{y}_0, y_0)$$

# Koopman Model Structure (Neuromancer)

**Goal:** Identify a linear dynamic system in a learned latent space using neural networks.

- **Encoder**  $\phi(x)$  maps real system state  $x$  to latent state  $z$ :

$$z_k = \phi(x_k)$$

- **Koopman Dynamics:** Linear evolution in latent space:

$$z_{k+1} = Az_k + Bu_k$$

- **Decoder**  $\psi(z)$  reconstructs output from latent state:

$$\hat{y}_k = \psi(z_k)$$

- **Linear Output (used for MPC):**

$$y_k = Cz_k$$

- Trained on **scaled data (StandardScaler)** using train + test together (global scaling).

# Koopman Output Decoding

How is the output predicted from the Koopman latent state  $z_k$ ?

- **Two decoding methods are trained:**

- ① **Decoder network**  $\psi(z_k)$ :

$$\hat{y}_k = \psi(z_k)$$

- Nonlinear neural network trained during identification
    - Used only in training and evaluation

- ② **Linear output map**  $C$  (learned during training):

$$\hat{y}_k = Cz_k$$

- Used in predictive control (MPC)
    - Simpler and more efficient for linear prediction
  - $\psi$  helps learning richer dynamics and interpretable states
  - $C$  gives a fast, low-complexity output for MPC control
- Koopman-MPC simulations in MATLAB use only the  $C$  matrix output

# Role of Koopman Matrices

The linear Koopman model:

$$z_{k+1} = Az_k + Bu_k, \quad \hat{y}_k = Cz_k + Du_k$$

- $A$  : State transition matrix. Describes how the latent state evolves without input:

$$z_{k+1} = Az_k$$

- $B$  : Input matrix. Models the influence of control input on the state:

$$z_{k+1} = Bu_k$$

- $C$  : Output matrix. Transforms latent state into observable output, This is the only matrix used during MPC decoding.:

$$\hat{y}_k = Cz_k$$

- $D$  : Direct input-to-output path.  $D = 0$  So there is no direct feedthrough — output depends only on state.

# Koopman Principle

## Nonlinear dynamics:

$$x_{k+1} = f(x_k, u_k)$$

## Koopman lifting:

- Map state to higher-dimensional latent space:

$$z_k = g(x_k)$$

- Apply linear dynamics in latent space:

$$z_{k+1} = A_K z_k + B_K u_k$$

- Reconstruct output:

$$\hat{y}_k = C_K z_k, x_{k+1} = f(x_k, u_k)$$

# Speculations

## **Why did we use the Strejc model?**

The Strejc model is a simple first-order system model used for approximating step responses. It is easy to identify from data using graphical methods and can be implemented in predictive control with minimal computational effort. It serves as a useful benchmark for comparison with more complex models like Koopman.

## **How did we obtain the gain and time constant?**

We used normalized step responses from experimental data. The steady-state gain  $K$  was calculated as the final value of the average normalized response, and the time constant  $T$  was estimated as the time it took the response to reach 63.2% of  $K$ .

# Speculations

## **What are the advantages and disadvantages of Strejc and Koopman models?**

- Strejc: simple, interpretable, easy to implement but less accurate for nonlinear behavior
- Koopman: better accuracy and generalization, handles nonlinearity, but more complex to train and requires more data

## **Why does the Koopman model have 10 states?**

The latent dimension was chosen based on model selection and empirical testing. It balances complexity and performance: a lower dimension might underfit, while a higher one could overfit or slow down computation.



# Speculations

**Is Koopman always better than Strejc?** Not necessarily. Koopman performs better on complex, nonlinear dynamics, but for simple processes, Strejc can be more appropriate. Also, Koopman requires more computational resources.

**What do the encoder and decoder do in the Koopman model?** The encoder maps the original measurements to a latent space where the dynamics become linear. The decoder reconstructs the output from this latent state. Only the linear mapping ( $C$  matrix) is used in control.

# Speculations

## **What is latent space in this context?**

It's a learned internal representation of the system state where dynamics are approximately linear. The neural network transforms the original data into this higher-dimensional space.

## **Why not control directly using the neural network output?**

Neural networks are hard to use in real-time control due to their nonlinearities. Koopman provides a linear model that is efficient for MPC. We use neural networks only during training.

# Speculations

## **Why do we only use $A$ , $B$ , $C$ matrices in MPC?**

Once trained, we discard the neural network and only use the linear state-space matrices. This makes the controller fast, deterministic, and interpretable.

## **Can the Koopman model be interpreted physically?**

The latent states do not have direct physical meaning. However, the model captures system behavior well. The mapping to observable output allows us to interpret the result even without interpreting latent variables.

## **What was the reference used in MPC?**

The reference was 0 in scaled coordinates, which corresponds to the average measured temperature in  $^{\circ}\text{C}$ . This ensures that the system regulates to the steady-state value of typical operation.

# Speculations

## **Why did we choose $Q_y = 10$ and $R_u = 1$ ?**

These weights prioritize output accuracy while penalizing input effort moderately. The ratio was chosen based on trial simulations and performance trade-offs. **What would happen with a longer prediction horizon?**

It may improve control smoothness but increase computational time. For our system,  $N = 40$  provided a good balance.

## **How fast is the MPC computation?**

The final controller is quadratic programming-based (via YALMIP and quadprog) and can run in real time for small systems like this one.

# Speculations

## **Are the performance differences between models significant?**

Yes. Koopman showed consistently better RMSE and objective function value across different initial conditions. However, Strejc was still acceptable in many cases.

## **Is Koopman robust to changing conditions?**

Since it's data-driven, it can generalize if trained on varied conditions. However, extrapolation outside the data range may fail. Robustness can be improved by retraining or augmenting data.

## **What would we change with more time?**

Test Koopman on the real hardware, train on more varied data (e.g. disturbances), and compare with other nonlinear modeling approaches like NARX or recurrent neural networks.